

LAPORAN PRAKTIKUM TEKNOLOGI IoT II
JARINGAN KABEL DAN KOMUNIKASI HTTP



Oleh:

NAMA : AKBAR BETHARA IMANI

NPM : 25782088

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027

Rangkaian Praktikum 1 — NodeMCU sebagai Web

Server

Sensor DHT dihubungkan melalui pin DATA/OUT/S ke pin D4 (GPIO 2), dengan VCC ke 3V3 dan GND ke GND NodeMCU. Kaki Anoda (kaki panjang) LED dihubungkan ke pin D6 (GPIO 12), sedangkan kaki Katoda (kaki pendek) LED dihubungkan ke salah satu kaki resistor 220 Ohm, lalu kaki resistor lainnya dihubungkan ke pin GND NodeMCU. Pin D6/GPIO 12 ini didefinisikan sebagai relayPin pada kode program.

Program Praktikum 1

Berikut kode program yang di-upload ke NodeMCU sebagai Web Server. Variabel ssid dan password diganti sesuai jaringan Wi-Fi yang digunakan.

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "brrrr";
const char* password = "12345678";

ESP8266WebServer server(80);

const byte dhtPin = 2;
const byte relayPin = 12;
DHT dht(dhtPin, DHT11);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top:
50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius:
8px; margin: 10px; cursor: pointer;}
```

```

        .btn-on { background-color: #4CAF50; color: white; border:
none; }
        .btn-off { background-color: #f44336; color: white; border:
none; }
        .sensor-box { font-size: 24px; font-weight: bold; }
    </style>
</head>
<body>
    <h1>ESP8266 Web Server</h1>
    <div class="sensor-box">
        <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
    </div>
    <h2>Kendali Relay</h2>
    <a href="/relay/on"><button class="btn-on">ON</button></a>
    <a href="/relay/off"><button class="btn-off">OFF</button></a>
</body>
</html>
)rawliteral";

```

```

void handleRoot() {
    String html = index_html;
    float t = dht.readTemperature();

    if (isnan(t)) {
        html.replace("%TEMPERATURE%", "--");
    } else {
        html.replace("%TEMPERATURE%", String(t));
    }

    server.send(200, "text/html", html);
}

```

```

void handleRelayOn() {
    digitalWrite(relayPin, HIGH);
    server.sendHeader("Location", "/");
    server.send(303);
}

```

```

void handleRelayOff() {
    digitalWrite(relayPin, LOW);
    server.sendHeader("Location", "/");
    server.send(303);
}

```

```

void setup() {
    Serial.begin(115200);
}

```

```

pinMode(relayPin, OUTPUT);
digitalWrite(relayPin, LOW);
dht.begin();

WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print("."); }
Serial.println("\nIP Address Server Anda: ");
Serial.println(WiFi.localIP());

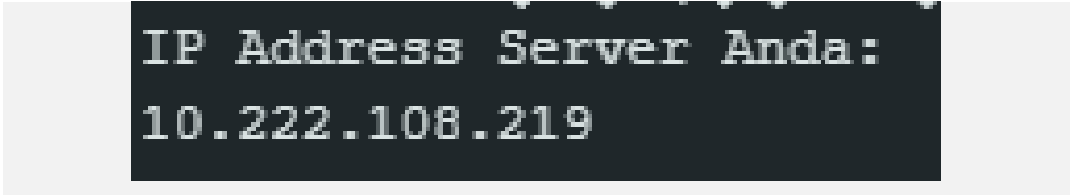
server.on("/", handleRoot);
server.on("/relay/on", handleRelayOn);
server.on("/relay/off", handleRelayOff);
server.begin();
}

void loop() {
  server.handleClient();
}

```

Hasil Pengujian Praktikum 1

Setelah kode di-upload, Serial Monitor dibuka untuk mencatat alamat IP yang diperoleh NodeMCU. Alamat IP tersebut kemudian diketikkan di browser pada perangkat yang terhubung ke Wi-Fi yang sama.



The image shows a screenshot of a web browser window. The address bar contains the IP address 10.222.108.219. The main content area of the browser displays the text "IP Address Server Anda:" followed by the IP address "10.222.108.219" on the next line. The text is in a monospaced font, typical of a terminal or serial monitor output.

Halaman web menampilkan suhu ruangan hasil pembacaan sensor DHT beserta dua tombol kendali (ON dan OFF).



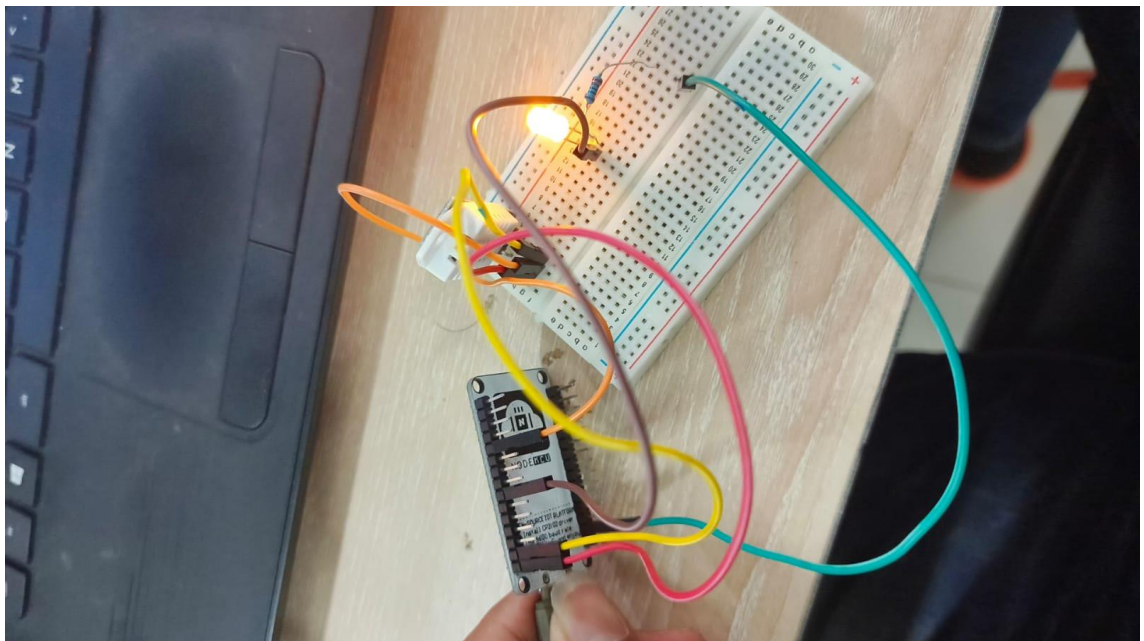
ESP8266 Web Server

Suhu Saat Ini: **30.40** Celcius

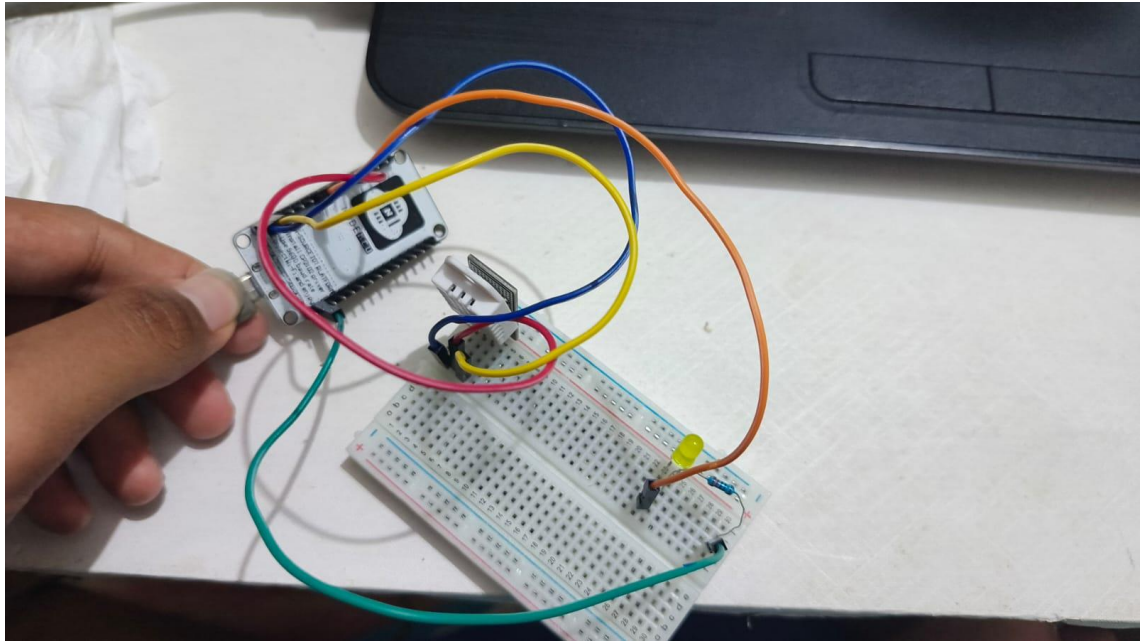
Kendali Relay



Tombol ON ditekan dan LED pada rangkaian menyala.



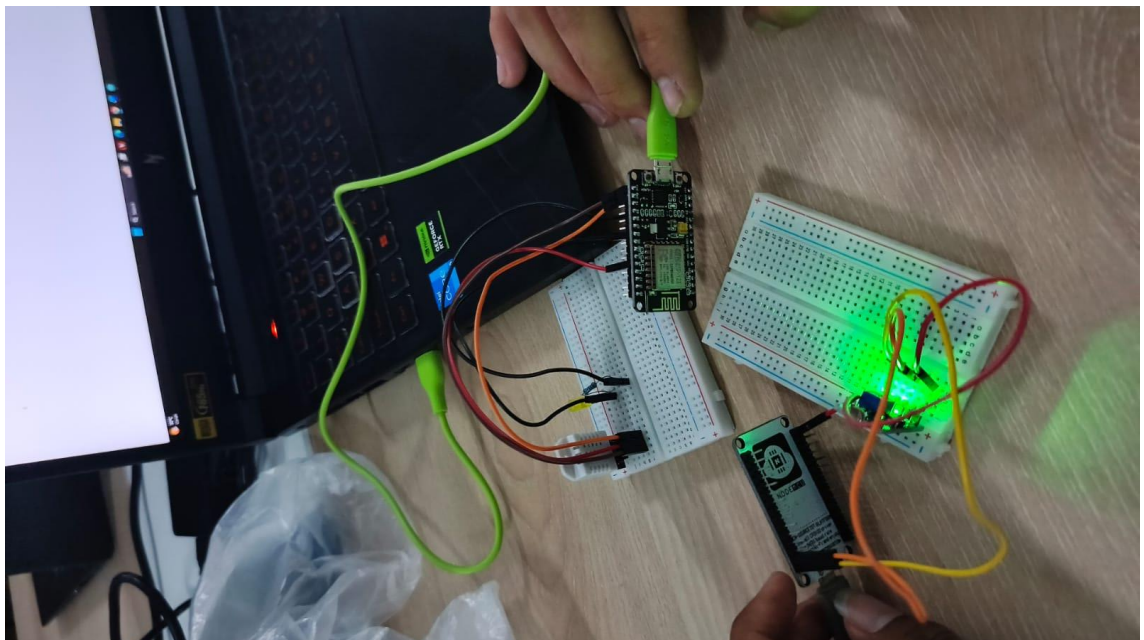
Tombol OFF ditekan dan LED pada rangkaian padam.



Rangkaian Praktikum 2 — NodeMCU sebagai HTTP

Client

Sensor LDR dan resistor 10k Ohm dirangkai sebagai pembagi tegangan, dengan jalur sinyal dihubungkan ke pin A0 pada NodeMCU kedua (Client).



Program Praktikum 2

Berikut kode program yang di-upload ke NodeMCU Client. Variabel serverName diganti dengan alamat IP NodeMCU Server (Praktikum 1) milik teman sekelompok, dan kedua perangkat menggunakan jaringan Wi-Fi yang sama.

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "brrrr";
const char* password = "12345678";

const char* serverName = "http://10.222.108.219/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500);
  Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  int ldrValue = analogRead(ldrPin);

  if ((WiFi.status() == WL_CONNECTED) && (ldrValue < 300)) {
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverName);
    int httpStatusCode = http.GET();

    Serial.print("HTTP Response code: ");
    Serial.println(httpStatusCode);

    http.end();
    delay(10000);
  }
  delay(2000);
}
```

Hasil Pengujian Praktikum 2

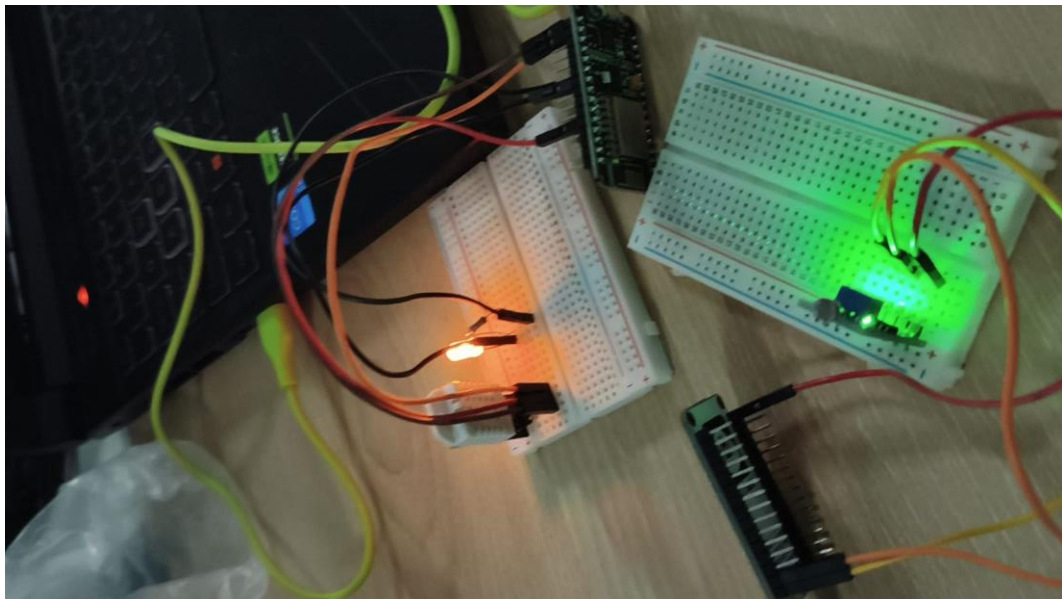
Setelah kode di-upload, Serial Monitor dibuka untuk memantau proses koneksi Wi-Fi dan kode respons HTTP yang dikirimkan.

Client Terhubung ke Wi-Fi!

LDR ditutup dengan tangan hingga kondisi gelap, sehingga Client mengirim permintaan GET ke Server.

HTTP Response Code : 303

LED pada NodeMCU Server (milik teman sekelompok) menyala secara otomatis sebagai respons dari permintaan Client.



Latihan 1 — Pemahaman Jaringan

Jawaban:

[Tuliskan alasan logis mengapa kedua praktikum di atas menggunakan mode Station (STA) dan bukan mode Access Point (AP), berdasarkan wawasan pada subbab 3.3.1.]

Latihan 2 — Modifikasi Antarmuka & Telemetry

a. Penambahan pembacaan kelembapan

Placeholder %HUMIDITY% disisipkan pada kerangka HTML di dalam tag `<div class="sensor-box">` sebagai berikut:

```
<div class="sensor-box">
```

```

    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
    <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
</div>

```

Fungsi `handleRoot()` diperbarui agar nilai kelembapan turut disisipkan menggunakan `dht.readHumidity()`:

```

void handleRoot() {
    String html = index_html;
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (isnan(t)) {
        html.replace("%TEMPERATURE%", "--");
    } else {
        html.replace("%TEMPERATURE%", String(t));
    }

    if (isnan(h)) {
        html.replace("%HUMIDITY%", "--");
    } else {
        html.replace("%HUMIDITY%", String(h));
    }

    server.send(200, "text/html", html);
}

```

b. Tombol dinamis tunggal (toggle button)

Placeholder `%RELAY_BUTTON%` disisipkan pada bagian kendali relay, menggantikan dua tombol terpisah:

```

<h2>Kendali Relay</h2>
%RELAY_BUTTON%

```

Pada fungsi `handleRoot()`, status pin relay diperiksa dengan `digitalRead(relayPin)` sebelum menentukan isi tombol yang akan ditampilkan:

```

void handleRoot() {
    String html = index_html;
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    html.replace("%TEMPERATURE%", isnan(t) ? "--" : String(t));
    html.replace("%HUMIDITY%", isnan(h) ? "--" : String(h));

    String tombol;
    if (digitalRead(relayPin) == LOW) {

```

```

    tombol = "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>";
  } else {
    tombol = "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>";
  }
  html.replace("%RELAY_BUTTON%", tombol);

  server.send(200, "text/html", html);
}

```



ESP8266 Web Server

Suhu Saat Ini: **30.20** Celcius

Kelembapan: **86.90** %

Kendali Relay

NYALAKAN (ON)

Latihan 3 — Analisis Kode Client

Jawaban:

kode respons negatif seperti -1 yang menandakan koneksi gagal.

HTTP Response Code : -1

Latihan 4 — Pengembangan Algoritma

Modifikasi conditional block agar Client mengirim permintaan GET hanya jika suhu yang dibaca sensor DHT melebihi 35°C, menggantikan kondisi berbasis LDR:

```
float suhu = dht.readTemperature();
```

```
if ((WiFi.status() == WL_CONNECTED) && (suhu > 35)) {
  WiFiClient client;
```

```

HTTPClient http;

http.begin(client, serverName);
int httpStatusCode = http.GET();

Serial.print("HTTP Response code: ");
Serial.println(httpStatusCode);

http.end();
delay(10000);
}

```

Tugas — Pengembangan Web Server Cerdas (Auto-Refresh)

Modifikasi dilakukan pada variabel Raw Literal `index_html` di sisi NodeMCU Server (hasil Latihan 2), dengan menyisipkan tag Meta Refresh di dalam cakupan tag `<head>` agar browser memuat ulang halaman secara otomatis setiap 5 detik:

```

<head>
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <meta http-equiv="refresh" content="5">
  <title>IoT Dashboard</title>
  ...
</head>

```

Evaluasi Analisis Arsitektur:

Metode Meta Refresh memiliki kelemahan karena browser memuat ulang seluruh halaman setiap 5 detik, bukan hanya memperbarui data sensor. Hal ini dapat menyebabkan layar berkedip atau flickering ketika halaman dimuat ulang. Metode ini juga menghasilkan permintaan HTTP secara berulang sehingga menggunakan bandwidth dan sumber daya NodeMCU lebih banyak. Selain itu, jika pengguna sedang menekan tombol ON atau OFF tepat ketika halaman melakukan refresh,

interaksi dapat terganggu karena halaman sedang dimuat ulang. Oleh karena itu, Meta Refresh dapat digunakan untuk pembaruan sederhana, tetapi kurang efisien untuk dashboard IoT yang membutuhkan pembaruan data secara real-time dan interaksi yang lebih responsif.